

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего
образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ

«Информатика и системы управления»

Кафедра

ИУ-6 «Компьютерные системы и сети»

Группа

«ИУ6-64Б »

ВЫЧИСЛИТЕЛЬНАЯ МАТЕМАТИКА

Отчет по домашнему заданию N1:

Прямые методы решения СЛАУ

Вариант N

Студент:

Скобелев В.М.

дата, подпись

Ф.И.О.

Преподаватель:

Орлова А. С.

дата, подпись

Ф.И.О.

Москва, 2025

Содержание

Введение	3
Исходные данные	4
Краткое описание методов	6
Реализация метода LU-разложения C++	8
Реализация метода Гаусса на C++	19
Реализация метода прогонки (Томаса) на C++	30

Введение

Цель домашней работы: изучения методов Гаусса, LU-разложения численного решения квадратной СЛАУ с невырожденной матрицей, оценка числа обусловленности матрицы и исследования его влияния на погрешность приближенного решения. Изучения метода прогонки решения СЛАУ с трехдиагональной матрицей.

Для достижения цели необходимо решить следующие задачи:

1. Реализовать на языке C/C++ указанные методы решения СЛАУ.
2. Провести решение двух заданных квадратных СЛАУ указанными методами, вычислить нормы невязок полученных приближенных решений, их предельные абсолютные и относительные погрешности (использовать 1-норму и ∞ -норму).
3. С использованием реализованных методов найти обратные матрицы для матриц квадратных систем, проверить выполнение равенств $A^{-1} \cdot A = E$
4. Для каждой системы оценить порядок числа обусловленности матрицы системы и сделать вывод о его влиянии на точность полученного приближенного решения и отвечающему ему невязку.
5. Провести решение СЛАУ с трехдиагональной матрицей реализованным методом прогонки и найти норму его невязки (использовать 1-норму и ∞ -норму).

Отчет должен содержать:

1. Постановку задачи и исходные данные.
2. Краткое описание реализуемых методов.
3. Текст программы.
4. Результаты расчетов (оформленных в виде таблицы).
5. Анализ полученных результатов (влияние числа обусловленности матрицы системы на точность полученного приближенного решения и отвечающему ему невязку).

Исходные данные

1. Хорошо обусловленная система

$$\left[\begin{array}{cccc|c} -150.6000 & -8.2900 & 7.3900 & 8.5700 & -484.3700 \\ 7.7000 & -77.0000 & -0.8900 & -2.6200 & -355.0300 \\ 4.3300 & 3.0300 & 146.8000 & -4.2800 & 1077.1400 \\ 8.7400 & -2.6000 & -1.2700 & -112.4000 & 566.3300 \end{array} \right] \quad \begin{array}{l} x_1 = 3 \\ x_2 = 5 \\ x_3 = 7 \\ x_4 = -5 \end{array}$$

2. Плохо обусловленная система

$$\left[\begin{array}{cccc|c} 1.8400 & 0.0400 & 7.0840 & -23.2920 & -7.1360 \\ 27.0000 & -0.0690 & 108.4760 & -345.3930 & -319.6040 \\ 5.4000 & -0.0100 & 21.6690 & -69.0570 & -62.6760 \\ 1.8000 & 0.0000 & 7.2000 & -23.0000 & -19.8000 \end{array} \right] \quad \begin{array}{l} x_1 = 5 \\ x_2 = 300 \\ x_3 = -4 \\ x_4 = 0 \end{array}$$

3. Трехдиагональная СЛАУ

$$\begin{pmatrix} b_1 & c_1 & & & & & \\ a_2 & b_2 & c_2 & & & & \\ & a_3 & b_3 & c_3 & & & \\ & & a_4 & b_4 & c_4 & & \\ & & & a_5 & b_5 & c_5 & \\ & & & & a_6 & b_6 & c_6 \\ & & & & & a_7 & b_7 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{pmatrix} = \begin{pmatrix} d_1 \\ d_2 \\ d_3 \\ d_4 \\ d_5 \\ d_6 \\ d_7 \end{pmatrix}$$

где коэффициенты равны:

Поддиагональ $a = (-1, 0, 2, 1, 1, 1)$

Диагональ $b = (84, 73, 52, 91, 66, 113, 74)$

Наддиагональ $c = (-1, 1, 0, 1, -2, 1)$

Правая часть $d = (17, 13, 10, 20, 13, 20, 14)$

Или в более наглядной форме:

$$\begin{pmatrix} 84 & -1 & & & & & \\ -1 & 73 & 1 & & & & \\ & 0 & 52 & 0 & & & \\ & & 2 & 91 & 1 & & \\ & & & 1 & 66 & -2 & \\ & & & & 1 & 113 & 1 \\ & & & & & 1 & 74 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{pmatrix} = \begin{pmatrix} 17 \\ 13 \\ 10 \\ 20 \\ 13 \\ 20 \\ 14 \end{pmatrix}$$

Краткое описание методов

Далее будет приведено краткое описание методов использованных для выполнения ДЗ1.

1. Метод Гаусса

- **Суть:** Последовательное исключение переменных приведением расширенной матрицы $[A|b]$ к треугольному виду

- **Реализация:**

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ 0 & a_{22}^{(1)} & \cdots & a_{2n}^{(1)} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & a_{nn}^{(n-1)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1^{(0)} \\ b_2^{(1)} \\ \vdots \\ b_n^{(n-1)} \end{pmatrix}$$

- **Особенности:**

- Выбор главного элемента (частичный поворот)
- Сложность: $O(n^3)$ операций
- Проверка на вырожденность: $\det(A) \approx 0$

2. LU-разложение

- **Теорема:** Для невырожденной матрицы $\exists!$ разложение $A = LU$, где:

$$L = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ l_{21} & 1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ l_{n1} & \cdots & l_{n,n-1} & 1 \end{pmatrix}, \quad U = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \ddots & \vdots \\ \vdots & \ddots & \ddots & u_{n-1,n} \\ 0 & \cdots & 0 & u_{nn} \end{pmatrix}$$

- **Алгоритм:**

1. Прямой ход: $Ly = b$
2. Обратный ход: $Ux = y$

3. Число обусловленности

- **Определение:** $\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$

- Нормы:

$$\|A\|_1 = \max_j \sum_{i=1}^n |a_{ij}|$$

$$\|A\|_\infty = \max_i \sum_{j=1}^n |a_{ij}|$$

- Интерпретация:

- $\text{cond}(A) \approx 1$ - хорошая обусловленность
- $\text{cond}(A) \gg 1$ - плохая обусловленность

4. Метод прогонки (Томаса)

- Для трехдиагональных систем:

$$\begin{pmatrix} b_1 & c_1 & 0 & \cdots & 0 \\ a_2 & b_2 & c_2 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & a_{n-1} & b_{n-1} & c_{n-1} \\ 0 & \cdots & 0 & a_n & b_n \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n-1} \\ x_n \end{pmatrix} = \begin{pmatrix} d_1 \\ d_2 \\ \vdots \\ d_{n-1} \\ d_n \end{pmatrix}$$

- Алгоритм:

1. Прямой ход (вычисление прогоночных коэффициентов):

$$\alpha_i = -\frac{c_i}{b_i + a_i \alpha_{i-1}}$$

$$\beta_i = \frac{d_i - a_i \beta_{i-1}}{b_i + a_i \alpha_{i-1}}$$

2. Обратный ход:

$$x_n = \beta_n, \quad x_i = \alpha_i x_{i+1} + \beta_i$$

- Сложность: $O(n)$ операций

5. Анализ погрешностей

- Метрики:

$$\text{Невязка : } \|r\| = \|b - A\hat{x}\|$$

$$\text{Абс. погрешность : } |\delta x| = |x - \hat{x}|$$

$$\text{Отн. погрешность : } \frac{|\delta x|}{|x|}$$

- Оценки:

$$\frac{|\delta x|}{|x|} \leq \text{cond}(A) \left(\frac{|\delta A|}{|A|} + \frac{|\delta b|}{|b|} \right)$$

Реализация метода LU-разложения C++

1. Структура программы

Программа реализует следующие численные методы:

- LU-разложение матрицы
- Решение СЛАУ методом LU-разложения
- Вычисление обратной матрицы
- Оценка числа обусловленности
- Анализ погрешностей решения

2. Исходный код

Листинг 1 – Основные функции программы

```
#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>
#include <stdexcept>

using namespace std;

// =====
// Вспомогательные функции для работы с векторами
// =====

/**
 * Вычисляет норму 1- вектора сумма ( абсолютных значений )
 */
double calculateVectorNorm1(const vector<double>& vec) {
    double sum = 0.0;
    for (double value : vec) sum += abs(value);
    return sum;
}
```



```

/**
 * Вычисляет норму - вектора максимальное ( абсолютное значение )
 */
double calculateVectorNormInf(const vector<double>& vec) {
    double maxValue = 0.0;
    for (double value : vec) maxValue = max(maxValue, abs(value));
    return maxValue;
}

/**
 * Вычисляет норму 1- матрицы максимальная ( сумма по столбцам )
 */
double calculateMatrixNorm1(const vector<vector<double>>& matrix) {
    double maxColSum = 0.0;
    int size = matrix.size();
    for (int col = 0; col < size; col++) {
        double colSum = 0.0;
        for (int row = 0; row < size; row++) {
            colSum += abs(matrix[row][col]);
        }
        maxColSum = max(maxColSum, colSum);
    }
    return maxColSum;
}

/**
 * Вычисляет норму - матрицы максимальная ( сумма по строкам )
 */
double calculateMatrixNormInf(const vector<vector<double>>& matrix) {
    double maxRowSum = 0.0;
    for (const auto& row : matrix) {
        double rowSum = 0.0;
        for (double value : row) rowSum += abs(value);
        maxRowSum = max(maxRowSum, rowSum);
    }
    return maxRowSum;
}

// =====
// Функции для работы с матрицами
// =====

```

```

/**
 * Выводит матрицу в удобочитаемом формате
 */
void printMatrix(const vector<vector<double>>& matrix, const string&
    label) {
    cout << label << ":\n";
    int size = matrix.size();
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            cout << setw(15) << setprecision(8) << scientific << matrix[i]
                [j];
        }
        cout << "\n";
    }
}

/**
 * Выполняет LU разложение матрицы
 */
void performLUdecomposition(const vector<vector<double>>& inputMatrix,
    vector<vector<double>>& lowerTriangular,
    vector<vector<double>>& upperTriangular) {
    int size = inputMatrix.size();
    lowerTriangular = vector<vector<double>>(size, vector<double>(size,
        0.0));
    upperTriangular = vector<vector<double>>(size, vector<double>(size,
        0.0));

    for (int i = 0; i < size; i++) {
        // Вычисляем верхнюю треугольную матрицу
        for (int k = i; k < size; k++) {
            double sum = 0.0;
            for (int j = 0; j < i; j++) {
                sum += lowerTriangular[i][j] * upperTriangular[j][k];
            }
            upperTriangular[i][k] = inputMatrix[i][k] - sum;
        }

        // Вычисляем нижнюю треугольную матрицу
        for (int k = i; k < size; k++) {
            if (i == k) {
                lowerTriangular[i][i] = 1.0;
            }
        }
    }
}

```

```

        }
        else {
            double sum = 0.0;
            for (int j = 0; j < i; j++) {
                sum += lowerTriangular[k][j] * upperTriangular[j][i];
            }
            lowerTriangular[k][i] = (inputMatrix[k][i] - sum) /
                upperTriangular[i][i];
        }
    }
}

/**
 * Решает систему уравнений методом LU разложения -
 */
vector<double> solveLinearSystem(const vector<vector<double>>&
    coefficients,
    const vector<double>& rightHandSide) {
    int size = coefficients.size();
    vector<vector<double>> L, U;
    performLUDecomposition(coefficients, L, U);

    vector<double> intermediate(size), solution(size);

    // Прямая подстановка (Ly = b)
    for (int i = 0; i < size; i++) {
        double sum = 0.0;
        for (int j = 0; j < i; j++) {
            sum += L[i][j] * intermediate[j];
        }
        intermediate[i] = rightHandSide[i] - sum;
    }

    // Обратная подстановка (Ux = y)
    for (int i = size - 1; i >= 0; i--) {
        double sum = 0.0;
        for (int j = i + 1; j < size; j++) {
            sum += U[i][j] * solution[j];
        }
        solution[i] = (intermediate[i] - sum) / U[i][i];
    }
}

```

```

        return solution;
    }

/**
 * Вычисляет обратную матрицу
 */
vector<vector<double>> computeInverseMatrix(const vector<vector<double>>&
    matrix) {
    int size = matrix.size();
    vector<vector<double>> inverse(size, vector<double>(size));
    vector<vector<double>> L, U;
    performLUdecomposition(matrix, L, U);

    for (int col = 0; col < size; col++) {
        vector<double> unitVector(size, 0.0);
        unitVector[col] = 1.0;

        vector<double> solution = solveLinearSystem(matrix, unitVector);

        for (int row = 0; row < size; row++) {
            inverse[row][col] = solution[row];
        }
    }

    return inverse;
}

/**
 * Умножает две матрицы
 */
vector<vector<double>> multiplyMatrices(const vector<vector<double>>& A,
    const vector<vector<double>>& B) {
    int size = A.size();
    vector<vector<double>> result(size, vector<double>(size, 0.0));

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            for (int k = 0; k < size; k++) {
                result[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

```

```

    }

    return result;
}

// =====
// Функции для анализа точности решения
// =====

/**
 * Вычисляет метрики точности решения
 */
void calculateSolutionAccuracy(const vector<vector<double>>& coefficients
    ,
    const vector<double>& rightHandSide,
    const vector<double>& computedSolution,
    const vector<double>& exactSolution,
    double& residualNorm1,
    double& residualNormInf,
    double& absoluteErrorNorm1,
    double& absoluteErrorNormInf,
    double& relativeErrorNorm1,
    double& relativeErrorNormInf) {

    int size = coefficients.size();
    vector<double> residual(size);
    vector<double> error(size);

    // Вычисляем невязку :  $r = b - Ax$ 
    for (int i = 0; i < size; i++) {
        double sum = 0.0;
        for (int j = 0; j < size; j++) {
            sum += coefficients[i][j] * computedSolution[j];
        }
        residual[i] = rightHandSide[i] - sum;
    }

    // Вычисляем погрешность :  $e = x - x_{exact}$ 
    for (int i = 0; i < size; i++) {
        error[i] = computedSolution[i] - exactSolution[i];
    }
}

```

```

    residualNorm1 = calculateVectorNorm1(residual);
    residualNormInf = calculateVectorNormInf(residual);
    absoluteErrorNorm1 = calculateVectorNorm1(error);
    absoluteErrorNormInf = calculateVectorNormInf(error);
    relativeErrorNorm1 = absoluteErrorNorm1 / calculateVectorNorm1(
        exactSolution);
    relativeErrorNormInf = absoluteErrorNormInf / calculateVectorNormInf(
        exactSolution);
}

// =====
// Основная функция для решения системы
// =====

void solveAndAnalyzeSystem(const vector<vector<double>>& augmentedMatrix,
    const vector<double>& exactSolution,
    const string& systemName) {
    int size = augmentedMatrix.size();
    vector<vector<double>> coefficients(size, vector<double>(size));
    vector<double> rightHandSide(size);

    // Разделяем расширенную матрицу на матрицу коэффициентов и правую часть
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            coefficients[i][j] = augmentedMatrix[i][j];
        }
        rightHandSide[i] = augmentedMatrix[i][size];
    }

    cout << "\n" << string(70, '=') << "\n";
    cout << systemName << "\n";
    cout << string(70, '=') << "\n";

    // 1. Вывод исходной системы
    cout << "\n1. Матрица коэффициентов :\n";
    printMatrix(coefficients, "A");
    cout << "\nВектор правой части b:\n";
    for (double value : rightHandSide) {
        cout << setw(15) << setprecision(8) << scientific << value << "\n";
    }
}

```

```

// 2. Решение системы
try {
    vector<double> solution = solveLinearSystem(coefficients,
        rightHandSide);
    cout << "\n2. Полученное решение : \n";
    for (int i = 0; i < size; i++) {
        cout << "x[" << i << "] = " << setw(15) << setprecision(8) <<
            solution[i]
            << " точное значение: " << exactSolution[i] << ") \n";
    }

// 3. Анализ точности решения
double resNorm1, resNormInf, absErrNorm1, absErrNormInf,
    relErrNorm1, relErrNormInf;
calculateSolutionAccuracy(coefficients, rightHandSide, solution,
    exactSolution,
    resNorm1, resNormInf, absErrNorm1, absErrNormInf,
    relErrNorm1, relErrNormInf);

cout << "\n3. Анализ точности решения : \n";
cout << " Невязка норма (1-): " << setw(15) << resNorm1 <<
    "\n";
cout << " Невязка норма (-): " << setw(15) << resNormInf
    << "\n";
cout << " Абсолютная погрешность (1): " << setw(15) << absErrNorm1
    << "\n";
cout << " Абсолютная погрешность (): " << setw(15) << absErrNormInf
    << "\n";
cout << " Относительная погрешность (1): " << setw(15) <<
    relErrNorm1 << "\n";
cout << " Относительная погрешность (): " << setw(15) <<
    relErrNormInf << "\n";

// 4. Вычисление и проверка обратной матрицы
vector<vector<double>> inverse = computeInverseMatrix(
    coefficients);
printMatrix(inverse, "\n4. Обратная матрица  $A^{-1}$ ");

// 5. Проверка  $A^{-1} * A = E$ 
vector<vector<double>> identityCheck = multiplyMatrices(inverse,
    coefficients);
double maxDeviation = 0.0;

```

```

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            double deviation = abs(identityCheck[i][j] - (i == j ?
                1.0 : 0.0));
            maxDeviation = max(maxDeviation, deviation);
        }
    }

    cout << "\n5. Проверка обратной матрицы : \n";
    cout << " Максимальное отклонение от единичной матрицы : " <<
        maxDeviation << "\n";

    // 6. Вычисление числа обусловленности
    double conditionNumber1 = calculateMatrixNorm1(coefficients) *
        calculateMatrixNorm1(inverse);
    double conditionNumberInf = calculateMatrixNormInf(coefficients)
        * calculateMatrixNormInf(inverse);

    cout << "\n6. Число обусловленности матрицы : \n";
    cout << " По норме 1-: cond1(A) = " << conditionNumber1 << "\n";
    cout << " По норме -: cond(A) = " << conditionNumberInf << "\n";

}

catch (const runtime_error& e) {
    cout << "Ошибка: " << e.what() << "\n";
}

cout << string(70, '=') << "\n";
}

// =====
// Главная функция программы
// =====

int main() {
    setlocale(LC_ALL, "Russian");
    cout << fixed << setprecision(8);

    // Хорошо обусловленная система
    vector<vector<double>> wellConditionedSystem = {
        {-150.6000, -8.2900, 7.3900, 8.5700, -484.3700},
        {7.7000, -77.0000, -0.8900, -2.6200, -355.0300},
        {4.3300, 3.0300, 146.8000, -4.2800, 1077.1400},
        {8.7400, -2.6000, -1.2700, -112.4000, 566.3300}
    };

```



```

};
vector<double> exactSolutionWell = { 3.0, 5.0, 7.0, -5.0 };

// Плохообусловленная система
vector<vector<double>> illConditionedSystem = {
    {1.8400, 0.0400, 7.0840, -23.2920, -7.1360},
    {27.0000, -0.0690, 108.4760, -345.3930, -319.6040},
    {5.4000, -0.0100, 21.6690, -69.0570, -62.6760},
    {1.8000, 0.0000, 7.2000, -23.0000, -19.8000}
};

vector<double> exactSolutionIll = { 5.0, 300.0, -4.0, 0.0 };

// Решение и анализ систем
solveAndAnalyzeSystem(wellConditionedSystem, exactSolutionWell,
    "АНАЛИЗ ХОРОШООБУСЛОВЛЕННОЙ СИСТЕМЫ ");
solveAndAnalyzeSystem(illConditionedSystem, exactSolutionIll,
    "АНАЛИЗ ПЛОХОООБУСЛОВЛЕННОЙ СИСТЕМЫ ");

return 0;
}

```

Таблица 1 – Функции программы и их назначение

Функция	Назначение
calculateVectorNorm1	Вычисляет 1-норму вектора (сумма абсолютных значений элементов)
calculateMatrixNormInf	Вычисляет ∞ -норму матрицы (максимальная сумма по строкам)
performLUDecomposition	Выполняет LU-разложение матрицы на нижнюю и верхнюю треугольные
solveLinearSystem	Решает СЛАУ методом LU-разложения (прямая и обратная подстановка)
computeInverseMatrix	Вычисляет обратную матрицу через решение систем с единичными векторами
calculateSolutionAccuracy	Оценивает точность решения через невязку и погрешности
solveAndAnalyzeSystem	Основная функция, объединяющая все этапы решения и анализа

3. Ключевые особенности реализации

- Использование STL-контейнеров (`vector`) для хранения матриц и векторов
- Чёткое разделение на модули (нормы, матричные операции, решение систем)
- Проверка на вырожденность матрицы при LU-разложении
- Вычисление числа обусловленности в различных нормах (1-норма и ∞ -норма)

- Подробный вывод результатов с оценкой погрешностей решения

Результаты работы программы

Таблица 2 – Результаты анализа хорошо обусловленной системы

Параметр	Значение	Ожидаемое значение
Решение системы		
x_1	3.00000000×10^0	3.00000000×10^0
x_2	5.00000000×10^0	5.00000000×10^0
x_3	7.00000000×10^0	7.00000000×10^0
x_4	-5.00000000×10^0	-5.00000000×10^0
Метрики точности		
Невязка (1-норма)	0.00000000×10^0	-
Невязка (∞ -норма)	0.00000000×10^0	-
Абс. погрешность (1)	$8.88178420 \times 10^{-16}$	-
Абс. погрешность (∞)	$8.88178420 \times 10^{-16}$	-
Отн. погрешность (1)	$4.44089210 \times 10^{-17}$	-
Отн. погрешность (∞)	$1.26882631 \times 10^{-16}$	-
Обратная матрица и проверка		
Макс. отклонение $A^{-1}A$ от I	$1.11022302 \times 10^{-16}$	0
Число обусловленности (1-норма)	2.44735073×10^0	-
Число обусловленности (∞ -норма)	2.42557144×10^0	-

Таблица 3 – Результаты анализа плохо обусловленной системы

Параметр	Значение	Ожидаемое значение
Решение системы		
x_1	5.00000000×10^0	5.00000000×10^0
x_2	3.00000000×10^2	3.00000000×10^2
x_3	-4.00000000×10^0	-4.00000000×10^0
x_4	$5.20472554 \times 10^{-12}$	0.00000000×10^0
Метрики точности		
Невязка (1-норма)	$5.86197757 \times 10^{-14}$	-
Невязка (∞ -норма)	$5.68434189 \times 10^{-14}$	-
Абс. погрешность (1)	$6.67812294 \times 10^{-9}$	-
Абс. погрешность (∞)	$3.84261511 \times 10^{-9}$	-
Отн. погрешность (1)	$2.16120484 \times 10^{-11}$	-
Отн. погрешность (∞)	$1.28087170 \times 10^{-11}$	-
Обратная матрица и проверка		
Макс. отклонение $A^{-1}A$ от E	$3.72529030 \times 10^{-9}$	0
Число обусловленности (1-норма)	3.74905765×10^8	-
Число обусловленности (∞ -норма)	4.38116483×10^8	-

Анализ результатов

- Для хорошо обусловленной системы:
 - Идеальная точность решения (нулевая невязка)

- Минимальные погрешности (порядка 10^{-16})
 - Отличная точность обратной матрицы (отклонение 10^{-16})
 - Низкие значения чисел обусловленности (≈ 2.4)
- Для **плохо обусловленной системы**:
 - Заметные погрешности решения (до 10^{-9})
 - Значительное отклонение при проверке обратной матрицы (10^{-9})
 - Чрезвычайно высокие числа обусловленности ($\approx 10^8$)
 - Появление артефактов вычислений в x_4 (5.2×10^{-12} вместо 0)

Реализация метода Гаусса на C++

1. Ключевые особенности реализации

- Использование STL-контейнеров (`vector<vector<double>`) для хранения матриц
- Проверка на вырожденность матрицы при обнаружении нулевого ведущего элемента
- Разделение на прямой и обратный ход метода

2. Исходный код

Листинг 2 – Исходный код программы

```
#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>
#include <stdexcept>
#include <algorithm>

using namespace std;

// =====
// Вспомогательные функции для работы с векторами
// =====

/**
 * Вычисляет норму 1- вектора сумма ( абсолютных значений )
 */
double calculateVectorNorm1(const vector<double>& vector) {
```

```

    double sum = 0.0;
    for (double value : vector) sum += abs(value);
    return sum;
}

/**
 * Вычисляет норму - вектора максимальное ( абсолютное значение )
 */
double calculateVectorNormInf(const vector<double>& vector) {
    double maxValue = 0.0;
    for (double value : vector) maxValue = max(maxValue, abs(value));
    return maxValue;
}

// =====
// Функции для работы с матрицами
// =====

/**
 * Выводит матрицу в научном формате
 */
void printMatrixScientific(const vector<vector<double>>& matrix, const
    string& label = "Matrix") {
    cout << label << ":\n";
    for (const auto& row : matrix) {
        for (double value : row) {
            cout << scientific << setw(12) << setprecision(8) << value <<
                " ";
        }
        cout << "\n";
    }
}

/**
 * Выводит вектор в научном формате
 */
void printVectorScientific(const vector<double>& vector, const string&
    label = "Vector") {
    cout << label << ":\n";
    for (double value : vector) {
        cout << scientific << setw(12) << setprecision(8) << value << "\n
        ";
    }
}

```

```

    }
}

/**
 * Решает систему линейных уравнений методом Гаусса с выбором главного элемента
 */
vector<double> solveSystemGauss(vector<vector<double>> augmentedMatrix) {
    int size = augmentedMatrix.size();

    // Прямой ход метода Гаусса
    for (int currentRow = 0; currentRow < size; currentRow++) {
        // Поиск строки с максимальным элементом в текущем столбце
        int maxRow = currentRow;
        for (int row = currentRow + 1; row < size; row++) {
            if (abs(augmentedMatrix[row][currentRow]) > abs(
                augmentedMatrix[maxRow][currentRow])) {
                maxRow = row;
            }
        }

        // Перестановка строк для выбора главного элемента
        swap(augmentedMatrix[currentRow], augmentedMatrix[maxRow]);

        // Проверка на вырожденность матрицы
        if (abs(augmentedMatrix[currentRow][currentRow]) < 1e-15) {
            throw runtime_error("Матрица системы вырождена ");
        }

        // Исключение переменных под текущей строкой
        for (int row = currentRow + 1; row < size; row++) {
            double eliminationFactor = augmentedMatrix[row][currentRow] /
                augmentedMatrix[currentRow][currentRow];
            for (int col = currentRow; col <= size; col++) {
                augmentedMatrix[row][col] -= eliminationFactor *
                    augmentedMatrix[currentRow][col];
            }
        }
    }

    // Обратный ход метода Гаусса
    vector<double> solution(size);
    for (int row = size - 1; row >= 0; row--) {

```

```

        solution[row] = augmentedMatrix[row][size] / augmentedMatrix[row][row];
        for (int upperRow = row - 1; upperRow >= 0; upperRow--) {
            augmentedMatrix[upperRow][size] -= augmentedMatrix[upperRow][row] * solution[row];
        }
    }

    return solution;
}

/**
 * Вычисляет обратную матрицу методом Гаусса-Жордана
 */
vector<vector<double>> computeMatrixInverse(const vector<vector<double>>& matrix) {
    int size = matrix.size();
    vector<vector<double>> augmentedMatrix(size, vector<double>(2 * size));

    // Создание расширенной матрицы [A|I]
    for (int row = 0; row < size; row++) {
        for (int col = 0; col < size; col++) {
            augmentedMatrix[row][col] = matrix[row][col];
        }
        augmentedMatrix[row][size + row] = 1.0;
    }

    // Прямой ход метода Гаусса
    for (int currentRow = 0; currentRow < size; currentRow++) {
        // Поиск главного элемента
        int maxRow = currentRow;
        for (int row = currentRow + 1; row < size; row++) {
            if (abs(augmentedMatrix[row][currentRow]) > abs(augmentedMatrix[maxRow][currentRow])) {
                maxRow = row;
            }
        }

        swap(augmentedMatrix[currentRow], augmentedMatrix[maxRow]);

        // Проверка на вырожденность

```

```

        if (abs(augmentedMatrix[currentRow][currentRow]) < 1e-15) {
            throw runtime_error("Матрица вырождена, обратной не существует");
        }

        // Нормировка текущей строки
        double pivot = augmentedMatrix[currentRow][currentRow];
        for (int col = currentRow; col < 2 * size; col++) {
            augmentedMatrix[currentRow][col] /= pivot;
        }

        // Исключение элементов в текущем столбце
        for (int row = 0; row < size; row++) {
            if (row != currentRow && abs(augmentedMatrix[row][currentRow]) > 1e-15) {
                double factor = augmentedMatrix[row][currentRow];
                for (int col = currentRow; col < 2 * size; col++) {
                    augmentedMatrix[row][col] -= factor * augmentedMatrix[
                        currentRow][col];
                }
            }
        }
    }

    // Извлечение обратной матрицы из расширенной
    vector<vector<double>> inverse(size, vector<double>(size));
    for (int row = 0; row < size; row++) {
        for (int col = 0; col < size; col++) {
            inverse[row][col] = augmentedMatrix[row][size + col];
        }
    }

    return inverse;
}

/**
 * Умножает две матрицы
 */
vector<vector<double>> multiplyMatrices(const vector<vector<double>>&
    firstMatrix,
    const vector<vector<double>>& secondMatrix) {
    int size = firstMatrix.size();
    vector<vector<double>> result(size, vector<double>(size, 0.0));

```

```

    for (int row = 0; row < size; row++) {
        for (int col = 0; col < size; col++) {
            for (int k = 0; k < size; k++) {
                result[row][col] += firstMatrix[row][k] * secondMatrix[k][col];
            }
        }
    }

    return result;
}

/**
 * Вычисляет число обусловленности матрицы
 */
pair<double, double> computeConditionNumbers(const vector<vector<double>>& matrix,
    const vector<vector<double>>& inverseMatrix) {
    double matrixNorm1 = 0.0, matrixNormInf = 0.0;
    double inverseNorm1 = 0.0, inverseNormInf = 0.0;
    int size = matrix.size();

    // Вычисление норм исходной матрицы
    for (int col = 0; col < size; col++) {
        double columnSum = 0.0;
        for (int row = 0; row < size; row++) {
            columnSum += abs(matrix[row][col]);
        }
        matrixNorm1 = max(matrixNorm1, columnSum);
    }

    for (int row = 0; row < size; row++) {
        double rowSum = 0.0;
        for (int col = 0; col < size; col++) {
            rowSum += abs(matrix[row][col]);
        }
        matrixNormInf = max(matrixNormInf, rowSum);
    }

    // Вычисление норм обратной матрицы
    for (int col = 0; col < size; col++) {

```



```

        double columnSum = 0.0;
        for (int row = 0; row < size; row++) {
            columnSum += abs(inverseMatrix[row][col]);
        }
        inverseNorm1 = max(inverseNorm1, columnSum);
    }

    for (int row = 0; row < size; row++) {
        double rowSum = 0.0;
        for (int col = 0; col < size; col++) {
            rowSum += abs(inverseMatrix[row][col]);
        }
        inverseNormInf = max(inverseNormInf, rowSum);
    }

    return { matrixNorm1 * inverseNorm1, matrixNormInf * inverseNormInf };
}

/**
 * Вычисляет максимальное отклонение от единичной матрицы
 */
double computeMaxIdentityDeviation(const vector<vector<double>>& matrix)
{
    double maxDeviation = 0.0;
    int size = matrix.size();

    for (int row = 0; row < size; row++) {
        for (int col = 0; col < size; col++) {
            double expectedValue = (row == col) ? 1.0 : 0.0;
            maxDeviation = max(maxDeviation, abs(matrix[row][col] -
                expectedValue));
        }
    }

    return maxDeviation;
}

// =====
// Основная функция для анализа системы
// =====

```

```

void analyzeLinearSystem(const vector<vector<double>>& augmentedMatrix,
    const vector<double>& exactSolution,
    const string& systemName) {
    int size = augmentedMatrix.size();
    vector<vector<double>> coefficientMatrix(size, vector<double>(size));
    vector<double> rightHandSide(size);

    // Разделение расширенной матрицы на матрицу коэффициентов и правую часть
    for (int row = 0; row < size; row++) {
        for (int col = 0; col < size; col++) {
            coefficientMatrix[row][col] = augmentedMatrix[row][col];
        }
        rightHandSide[row] = augmentedMatrix[row][size];
    }

    cout << "\n" << string(70, '=') << "\n";
    cout << systemName << "\n";
    cout << string(70, '=') << "\n\n";

    // 1. Вывод исходной системы
    cout << "1. Матрица коэффициентов системы :\n";
    printMatrixScientific(coefficientMatrix);
    cout << "\nВектор правых частей :\n";
    printVectorScientific(rightHandSide);
    cout << "\n";

    // 2. Решение системы
    cout << "2. Решение системы :\n";
    try {
        vector<double> computedSolution = solveSystemGauss(
            augmentedMatrix);

        // Вывод решения с сравнением с точным решением
        for (int i = 0; i < size; i++) {
            cout << "x[" << i << "] = " << scientific << setw(12) <<
                setprecision(8) << computedSolution[i]
                << " точное(" << scientific << setw(12) << setprecision
                (8) << exactSolution[i] << ")\n";
        }

        // 3. Вычисление метрики точности
        vector<double> residualVector(size);
    }
}

```

```

vector<double> errorVector(size);

// Вычисление невязки и погрешности
for (int row = 0; row < size; row++) {
    double sum = 0.0;
    for (int col = 0; col < size; col++) {
        sum += coefficientMatrix[row][col] * computedSolution[col];
    }
    residualVector[row] = rightHandSide[row] - sum;
    errorVector[row] = computedSolution[row] - exactSolution[row];
}

cout << "\n3. Метрика точности решения : \n";
cout << " Невязка норма (1-): " << scientific <<
    calculateVectorNorm1(residualVector) << "\n";
cout << " Невязка норма (-): " << scientific <<
    calculateVectorNormInf(residualVector) << "\n";
cout << " Абсолютная погрешность (1): " << scientific <<
    calculateVectorNorm1(errorVector) << "\n";
cout << " Абсолютная погрешность (): " << scientific <<
    calculateVectorNormInf(errorVector) << "\n";
cout << " Относительная погрешность (1): " << scientific <<
    calculateVectorNorm1(errorVector) / calculateVectorNorm1(
        exactSolution) << "\n";
cout << " Относительная погрешность (): " << scientific <<
    calculateVectorNormInf(errorVector) / calculateVectorNormInf(
        exactSolution) << "\n";

// 4. Вычисление и анализ обратной матрицы
try {
    vector<vector<double>> inverseMatrix = computeMatrixInverse(
        coefficientMatrix);
    cout << "\n4. Обратная матрица : \n";
    printMatrixScientific(inverseMatrix, "A^{-1}");

    // 5. Проверка точности обратной матрицы
    vector<vector<double>> identityCheck = multiplyMatrices(
        inverseMatrix, coefficientMatrix);
    double maxDeviation = computeMaxIdentityDeviation(
        identityCheck);
}

```

```

    cout << "\n5. Проверка  $A^{-1} * A = I$ :\n";
    cout << " Максимальноеотклонение : " << scientific <<
        maxDeviation << "\n";

    // 6. Вычислениечислаобусловленности
    auto [cond1, condInf] = computeConditionNumbers(
        coefficientMatrix, inverseMatrix);
    cout << "\n6. Числообусловленностиматрицы :\n";
    cout << " Пономре 1-: " << scientific << cond1 << "\n";
    cout << " Пономре -: " << scientific << condInf << "\n";

}

catch (const runtime_error& e) {
    cout << "\nОшибкаn привычисленииобратнойматрицы : " << e.what()
        << "\n";
}

}

catch (const runtime_error& e) {
    cout << "Ошибка прирешенииисистемы : " << e.what() << "\n";
}

}

// =====
// Главнаяфункцияпрограммы
// =====

int main() {
    // Установкалокалииточностивывода
    setlocale(LC_ALL, "Russian");
    cout << fixed << setprecision(8);

    // Хорошообусловленнаясистема
    vector<vector<double>> wellConditionedSystem = {
        {-150.6000, -8.2900, 7.3900, 8.5700, -484.3700},
        {7.7000, -77.0000, -0.8900, -2.6200, -355.0300},
        {4.3300, 3.0300, 146.8000, -4.2800, 1077.1400},
        {8.7400, -2.6000, -1.2700, -112.4000, 566.3300}
    };

    vector<double> exactSolutionWell = { 3.0, 5.0, 7.0, -5.0 };

    // Плохообусловленнаясистема

```

```

vector<vector<double>> illConditionedSystem = {
    {1.8400, 0.0400, 7.0840, -23.2920, -7.1360},
    {27.0000, -0.0690, 108.4760, -345.3930, -319.6040},
    {5.4000, -0.0100, 21.6690, -69.0570, -62.6760},
    {1.8000, 0.0000, 7.2000, -23.0000, -19.8000}
};

vector<double> exactSolutionIll = { 5.0, 300.0, -4.0, 0.0 };

// Анализ систем
analyzeLinearSystem(wellConditionedSystem, exactSolutionWell,
    "АНАЛИЗ ХОРОШООБУСЛОВЛЕННОЙ СИСТЕМЫ ");
analyzeLinearSystem(illConditionedSystem, exactSolutionIll,
    "АНАЛИЗ ПЛОХОООБУСЛОВЛЕННОЙ СИСТЕМЫ ");

return 0;
}

```

Таблица 4 – Функции метода Гаусса и их назначение

Функция	Назначение
<code>solveSystemGauss</code>	Основная функция, реализующая метод Гаусса с выбором главного элемента
<code>computeMatrixInverse</code>	Вычисление обратной матрицы методом Гаусса-Жордана
<code>multiplyMatrices</code>	Умножение матриц для проверки корректности обратной матрицы
<code>computeConditionNumbers</code>	Вычисление чисел обусловленности матрицы
<code>analyzeLinearSystem</code>	Комплексный анализ системы (решение + вычисление метрик)

Результаты работы программы

Анализ результатов

- Для **хорошо обусловленной системы**:
 - Наблюдается высокая точность решения (погрешности порядка 10^{-15})
 - Малое отклонение при проверке обратной матрицы (10^{-16})
 - Низкие значения чисел обусловленности (≈ 2.4)
- Для **плохо обусловленной системы**:
 - Заметно повышенные погрешности решения (до 10^{-10})
 - Значительное отклонение при проверке обратной матрицы (10^{-8})
 - Чрезвычайно высокие числа обусловленности ($\approx 10^8$)
 - Появление артефактов вычислений (ненулевое значение для x_4)

Таблица 5 – Результаты анализа хорошо обусловленной системы

Параметр	Значение	Ожидаемое значение
Решение системы		
x_1	3.00000000×10^0	3.00000000×10^0
x_2	5.00000000×10^0	5.00000000×10^0
x_3	7.00000000×10^0	7.00000000×10^0
x_4	-5.00000000×10^0	-5.00000000×10^0
Метрики точности		
Невязка (1-норма)	$1.70530257 \times 10^{-13}$	-
Невязка (∞ -норма)	$1.13686838 \times 10^{-13}$	-
Абс. погрешность (1)	$2.66453526 \times 10^{-15}$	-
Абс. погрешность (∞)	$8.88178420 \times 10^{-16}$	-
Отн. погрешность (1)	$1.33226763 \times 10^{-16}$	-
Отн. погрешность (∞)	$1.26882631 \times 10^{-16}$	-
Обратная матрица и проверка		
Макс. отклонение $A^{-1}A$ от I	$1.11022302 \times 10^{-16}$	0
Число обусловленности (1-норма)	2.44735073×10^0	-
Число обусловленности (∞ -норма)	2.42557144×10^0	-

Реализация метода прогонки (Томаса) на C++

1. Ключевые особенности реализации

- Специализирован для решения трёхдиагональных систем уравнений
- Эффективная сложность $O(n)$ по сравнению с $O(n^3)$ у общего случая
- Использование трёх векторов для хранения коэффициентов (поддиагональ, диагональ, наддиагональ)
- Состоит из прямого и обратного хода
- Минимальные требования к памяти - только 4 вектора
- Устойчивость при условии диагонального преобладания

2. Исходный код

Листинг 3 – Реализация метода прогонки

```

1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4 #include <algorithm>
5 #include <iomanip>
6
7 using namespace std;
```

Таблица 6 – Результаты анализа плохо обусловленной системы

Параметр	Значение	Ожидаемое значение
Решение системы		
x_1	5.00000000×10^0	5.00000000×10^0
x_2	3.00000000×10^2	3.00000000×10^2
x_3	-4.00000000×10^0	-4.00000000×10^0
x_4	$-1.14436099 \times 10^{-12}$	0.00000000×10^0
Метрики точности		
Невязка (1-норма)	$1.86517468 \times 10^{-14}$	-
Невязка (∞ -норма)	$1.42108547 \times 10^{-14}$	-
Абс. погрешность (1)	$8.51061831 \times 10^{-10}$	-
Абс. погрешность (∞)	$4.86920726 \times 10^{-10}$	-
Отн. погрешность (1)	$2.75424541 \times 10^{-12}$	-
Отн. погрешность (∞)	$1.62306909 \times 10^{-12}$	-
Обратная матрица и проверка		
Макс. отклонение $A^{-1}A$ от E	$7.45058060 \times 10^{-9}$	0
Число обусловленности (1-норма)	3.74905765×10^8	-
Число обусловленности (∞ -норма)	4.38116483×10^8	-

Таблица 7 – Функции метода прогонки и их назначение

Функция	Назначение
thomas_algorithm	Основная функция, реализующая метод прогонки
compute_residual	Вычисление невязки решения
compute_1_norm	Вычисление 1-нормы вектора
compute_inf_norm	Вычисление ∞ -нормы вектора
print_tridiagonal_matrix	Вывод трёхдиагональной матрицы

```

8
9 // Функция для решения трёхдиагональной системы методом прогонки
10 vector<double> thomas_algorithm(const vector<double>& a, const vector<
    double>& b,
11     const vector<double>& c, const vector<double>& d) {
12     int n = b.size();
13     vector<double> alpha(n - 1), beta(n);
14     vector<double> x(n);
15
16     // Прямой ход прогонки
17     alpha[0] = -c[0] / b[0];
18     beta[0] = d[0] / b[0];
19
20     for (int i = 1; i < n - 1; ++i) {
21         double denominator = a[i - 1] * alpha[i - 1] + b[i];
22         alpha[i] = -c[i] / denominator;
23         beta[i] = (d[i] - a[i - 1] * beta[i - 1]) / denominator;
24     }

```

```

25
26 // Последний шаг прямого хода
27 beta[n - 1] = (d[n - 1] - a[n - 2] * beta[n - 2]) / (a[n - 2] * alpha
    [n - 2] + b[n - 1]);
28
29 // Обратный ход прогонки
30 x[n - 1] = beta[n - 1];
31 for (int i = n - 2; i >= 0; --i) {
32     x[i] = alpha[i] * x[i + 1] + beta[i];
33 }
34
35 return x;
36 }
37
38 // Функция для вычисления невязки  $A \cdot x - d$ 
39 vector<double> compute_residual(const vector<double>& a, const vector<
    double>& b,
40     const vector<double>& c, const vector<double>& d,
41     const vector<double>& x) {
42     int n = b.size();
43     vector<double> residual(n);
44
45     residual[0] = b[0] * x[0] + c[0] * x[1] - d[0];
46     for (int i = 1; i < n - 1; ++i) {
47         residual[i] = a[i - 1] * x[i - 1] + b[i] * x[i] + c[i] * x[i + 1]
            - d[i];
48     }
49     residual[n - 1] = a[n - 2] * x[n - 2] + b[n - 1] * x[n - 1] - d[n -
        1];
50
51     return residual;
52 }
53
54 // Функция для вычисления нормы  $1$ - вектора
55 double compute_1_norm(const vector<double>& v) {
56     double norm = 0.0;
57     for (double val : v) {
58         norm += abs(val);
59     }
60     return norm;
61 }
62

```



```

63 // Функция для вычисления нормы - вектора
64 double compute_inf_norm(const vector<double>& v) {
65     double norm = 0.0;
66     for (double val : v) {
67         norm = max(norm, abs(val));
68     }
69     return norm;
70 }
71
72 // Функция для вывода трёхдиагональной матрицы
73 void print_tridiagonal_matrix(const vector<double>& a, const vector<
double>& b, const vector<double>& c) {
74     int n = b.size();
75     cout << "Трёхдиагональная матрица A:\n";
76     for (int i = 0; i < n; ++i) {
77         for (int j = 0; j < n; ++j) {
78             if (j == i) {
79                 cout << setw(6) << b[i] << " ";
80             }
81             else if (j == i - 1 && i > 0) {
82                 cout << setw(6) << a[i - 1] << " ";
83             }
84             else if (j == i + 1 && i < n - 1) {
85                 cout << setw(6) << c[i] << " ";
86             }
87             else {
88                 cout << setw(6) << 0 << " ";
89             }
90         }
91         cout << endl;
92     }
93 }
94
95 int main() {
96     setlocale(LC_ALL, "Russian");
97     // Исходные данные
98     vector<double> a = { -1, 0, 2, 1, 1, 1 }; // поддиагональ начиная (
        со 2-го элемента )
99     vector<double> b = { 84, 73, 52, 91, 66, 113, 74 }; // диагональ
100    vector<double> c = { -1, 1, 0, 1, -2, 1 }; // наддиагональ без (
        последнего элемента )
101    vector<double> d = { 17, 13, 10, 20, 13, 20, 14 }; // правая часть

```

```

102
103 // Вывод исходной системы
104 cout << "===== \n";
105 cout << " РЕШЕНИЕ СЛАУ МЕТОДОМ ПРОГОНКИ \n";
106 cout << "===== \n\n";
107
108 print_tridiagonal_matrix(a, b, c);
109
110 cout << "\n Вектор n правой части d: \n";
111 for (double val : d) {
112     cout << val << " ";
113 }
114 cout << "\n\n";
115
116 // Решение системы
117 vector<double> x = thomas_algorithm(a, b, c, d);
118
119 // Вычисление невязки
120 vector<double> residual = compute_residual(a, b, c, d, x);
121
122 // Вычисление норм невязки
123 double residual_1_norm = compute_1_norm(residual);
124 double residual_inf_norm = compute_inf_norm(residual);
125
126 // Вывод результатов
127 cout << "Решение системы x: \n";
128 for (int i = 0; i < x.size(); ++i) {
129     cout << "x[" << i << "] = " << x[i] << endl;
130 }
131
132
133 cout << "\n Вектор n невязки r = A*x - d: \n";
134 for (int i = 0; i < residual.size(); ++i) {
135     cout << "r[" << i << "] = " << scientific << residual[i] << endl;
136 }
137
138 cout << "\n Нормы невязки: \n";
139 cout << "норма 1-: " << scientific << residual_1_norm << endl;
140 cout << "норма -: " << scientific << residual_inf_norm << endl;
141 return 0;
142 }

```

3. Анализ результатов

Метод прогонки демонстрирует:

- Высокую эффективность для трёхдиагональных систем
- Точность решения сравнимую с методом Гаусса

4. Результаты работы метода прогонки

Таблица 8 – Результаты решения трёхдиагональной системы методом прогонки

Параметр	Значение	Ожидаемое значение
Решение системы		
x_0	$2.04503000 \times 10^{-1}$	-
x_1	$1.78249000 \times 10^{-1}$	-
x_2	$1.92308000 \times 10^{-1}$	-
x_3	$2.13367000 \times 10^{-1}$	-
x_4	$1.98997000 \times 10^{-1}$	-
x_5	$1.73577000 \times 10^{-1}$	-
x_6	$1.86844000 \times 10^{-1}$	-
Метрики точности		
Невязка (1-норма)	$7.10542700 \times 10^{-15}$	-
Невязка (∞ -норма)	$3.55271400 \times 10^{-15}$	-
Абс. погрешность (1)	-	-
Абс. погрешность (∞)	-	-
Отн. погрешность (1)	-	-
Отн. погрешность (∞)	-	-

Анализ результатов

- Для всех компонент решения получены значения в диапазоне $[0.17, 0.21]$
- Наблюдается исключительно малая невязка ($\sim 10^{-15}$)
- Отсутствие данных об обусловленности связано с особенностью метода прогонки, который не требует вычисления обратной матрицы
- Результаты подтверждают высокую точность и устойчивость метода для данной системы